

Trabajo Práctico N° 1

Redes Neuronales Feed Forward y Convolucionales

Aprendizaje Automático II
Tecnatura Universitaria en Inteligencia Artificial
FCEIA - UNR

Instancia	Fecha
Presentación del enunciado	martes 15 de septiembre
Entrega	viernes 9 de octubre, hasta las 23:59 hs
Defensas	lunes 19 de octubre y martes 20 de octubre

Introducción

En este trabajo práctico el grupo deberá diseñar, entrenar y evaluar redes neuronales aplicadas a cuatro problemas de distinta naturaleza. Los tres primeros se resuelven con redes de tipo *feed forward* (clasificación binaria, clasificación multiclase y regresión) y el cuarto con una red neuronal convolucional entrenada desde cero. El objetivo es consolidar los conceptos teóricos estudiados en las unidades de la materia a través de su implementación concreta sobre datos reales, tomando decisiones de arquitectura, preprocesamiento y optimización debidamente justificadas.

Consideraciones generales

- **Reproducibilidad:** fijar una semilla aleatoria al inicio de cada experimento (Python, NumPy y el framework de *deep learning* utilizado) para que los resultados sean reproducibles.
- **División del dataset:** trabajar siempre con tres conjuntos separados: **train**, **val** y **test**. El conjunto de entrenamiento es el único que el modelo ve durante el ajuste de pesos; el de validación se usa para monitorear el entrenamiento, comparar experimentos y tomar decisiones de diseño; el de test se usa **una sola vez**, al final, para reportar las métricas definitivas. Usar validación como criterio de selección y reportar test evita el sobreajuste implícito al conjunto de evaluación.
- **Checkpoints:** se recomienda guardar el estado del modelo periódicamente durante los entrenamientos más costosos y aprovechar la funcionalidad de *resume* para retomar desde el último *checkpoint* en caso de interrupción.

1. Problema 1: Clasificación Binaria

Objetivo

Construir un modelo de red neuronal que prediga si un paciente tiene alto riesgo de padecer un accidente cerebrovascular (ACV) o no, a partir de sus datos clínicos y demográficos.

Dataset

- **Tipo de problema:** Clasificación Binaria
- **Tamaño:** 5 110 pacientes $\times$ 12 características

Variable objetivo

Característica	Tipo	Descripción
stroke	Binario (Entero)	1: alto riesgo de ACV; 0: sin antecedentes.

Variables predictoras

Característica	Tipo	Descripción
Variables demográficas		
id	Entero	Identificador único del paciente.
gender	Categorico	Sexo biológico del paciente.
age	Numérico (Float)	Edad del paciente en años. Predictor fuerte.
ever_married	Binario (String)	Indica si el paciente ha estado casado.
Variables de historial médico		
hypertension	Binario (Entero)	Presencia de hipertensión arterial.
heart_disease	Binario (Entero)	Presencia de enfermedad cardíaca.
Variables de estilo de vida y entorno		
work_type	Categorico	Tipo de ocupación laboral.
Residence_type	Categorico	Área de residencia del paciente.
smoking_status	Categorico	Historial de tabaquismo.
Mediciones clínicas		
avg_glucose_level	Numérico (Float)	Nivel promedio de glucosa en sangre.
bmi	Numérico (Float)	Índice de Masa Corporal. Contiene valores faltantes que requieren imputación.

Consignas

- Análisis Exploratorio de Datos (EDA).** Realizar un análisis descriptivo del dataset. Visualizar la distribución de la variable objetivo e identificar el grado de desbalance de clases. Examinar la correlación de las variables predictoras con `stroke` mediante gráficos adecuados (histogramas, boxplots, mapas de calor).
- Preprocesamiento.**
 - Eliminar la columna `id` del dataset.
 - Tratar los valores faltantes; justificar la estrategia de imputación elegida.
 - Codificar las variables categóricas mediante la técnica que se considere apropiada. Justificar su elección.
 - Escalar las variables numéricas continuas. Aplicar el escalado **después** de realizar la partición *train/test* para evitar *data leakage*.
 - Aplicar alguna estrategia para mitigar el desbalance de clases (sobremuestreo, submuestreo u otra). Justificar la decisión.
- Diseño e implementación de la red neuronal.** Diseñar un MLP para clasificación binaria. Definir y justificar:
 - Número de capas ocultas y cantidad de neuronas por capa.
 - Función de activación en capas ocultas y en la capa de salida.
 - Función de costo.

- d) Optimizador (Mini-Batch Gradient Descent, Adam) y tasa de aprendizaje elegidos.
 - e) Técnica de regularización (L1, L2, Dropout, Early Stopping).
4. **Entrenamiento y curvas de aprendizaje.** Entrenar el modelo y visualizar las curvas de pérdida y de la métrica principal en los conjuntos de entrenamiento y validación. Identificar indicios de *overfitting* o *underfitting*.
 5. **Evaluación del modelo.** Dado el fuerte desbalance del dataset, reportar las siguientes métricas sobre el conjunto de test: *Accuracy*, Precisión, Recall, F1-Score y AUC-ROC. Justificar cuál es la mejor métrica para evaluar el modelo y cuál puede ser una métrica engañosa en este caso.
 6. **Matriz de Confusión.** Visualizar la Matriz de Confusión e interpretar los resultados. Analizar específicamente el impacto de los falsos negativos en un contexto clínico.
 7. **Análisis y conclusiones.** Discutir las limitaciones del modelo, las decisiones de diseño que resultaron más críticas y posibles mejoras futuras.

2. Problema 2: Clasificación Multiclase

Objetivo

Construir un modelo de red neuronal capaz de reconocer dígitos escritos a mano (0 al 9) a partir de imágenes en escala de grises.

Dataset

- **Tipo de problema:** Clasificación Multiclase (10 clases)
- **Archivos:** los datos se encuentran en la carpeta PROBLEMA 2 en formato binario IDX.

Archivo	Muestras	Dimensiones por muestra
train-images.idx3-ubyte	60 000	28×28 píxeles (uint8)
train-labels.idx1-ubyte	60 000	1 etiqueta (0–9)
t10k-images.idx3-ubyte	10 000	28×28 píxeles (uint8)
t10k-labels.idx1-ubyte	10 000	1 etiqueta (0–9)

Variable objetivo

Dígito manuscrito representado como un entero en el rango $[0, 9]$.

Variables predictoras

Cada imagen está compuesta por $28 \times 28 = 784$ píxeles en escala de grises (valores enteros sin signo en el rango $[0, 255]$). Para ser usada como entrada de un MLP, la imagen debe ser aplanada a un vector de 784 dimensiones.

2.1. Guía de procesamiento del dataset MNIST

Los archivos de MNIST están en formato binario IDX y **no pueden abrirse como imágenes estándar** (JPEG, PNG, etc.). A continuación se presentan los pasos esenciales para cargarlos correctamente.

Paso 1: Leer los archivos binarios IDX

Los enteros del encabezado están en formato *big-endian*; se usa '`>IIII`' para desempaquetarlos.

```

1 import struct
2 import numpy as np
3 import os
4
5 def load_mnist_images(filepath):
6     with open(filepath, 'rb') as f:
7         # Leer encabezado: numero magico, cantidad, filas, columnas
8         magic, num_images, rows, cols = struct.unpack('>IIII', f.read(16))
9         assert magic == 2051, f"Numero magico invalido: {magic}"
10        # Leer todos los pixeles y convertir a numpy
11        images = np.frombuffer(f.read(), dtype=np.uint8)
12        images = images.reshape(num_images, rows, cols)
13    return images
14
15 def load_mnist_labels(filepath):
16     with open(filepath, 'rb') as f:
17         # Leer encabezado: numero magico y cantidad de etiquetas
18         magic, num_labels = struct.unpack('>II', f.read(8))
19         assert magic == 2049, f"Numero magico invalido: {magic}"
20        labels = np.frombuffer(f.read(), dtype=np.uint8)
21    return labels
22
23 base_path = r"DATASETS/PROBLEMA 2"
24 X_train = load_mnist_images(os.path.join(base_path, "train-images.idx3-
        ubyte"))
25 y_train = load_mnist_labels(os.path.join(base_path, "train-labels.idx1-
        ubyte"))
26 X_test = load_mnist_images(os.path.join(base_path, "t10k-images.idx3-ubyte
        "))
27 y_test = load_mnist_labels(os.path.join(base_path, "t10k-labels.idx1-ubyte
        "))
28
29 print(f"X_train: {X_train.shape}") # (60000, 28, 28)
30 print(f"X_test: {X_test.shape}") # (10000, 28, 28)

```

Listing 1: Lectura de imágenes MNIST en formato IDX

Paso 2: Aplanar las imágenes para la entrada del MLP

A diferencia de las CNN, los MLP requieren vectores unidimensionales. Cada matriz de 28×28 se convierte en un vector de 784 valores, que se corresponde con el número de neuronas en la capa de entrada.

```

1 # (60000, 28, 28) -> (60000, 784)
2 X_train_flat = X_train.reshape(X_train.shape[0], -1)
3 X_test_flat = X_test.reshape(X_test.shape[0], -1)

```

```

4
5 print(f"X_train_flat: {X_train_flat.shape}") # (60000, 784)
6 print(f"X_test_flat: {X_test_flat.shape}") # (10000, 784)

```

Listing 2: Aplanamiento de imágenes

Paso 3: Normalizar los valores de los píxeles

Los píxeles se encuentran en el rango $[0, 255]$ (uint8). Es necesario convertirlos a float32 y escalarlos al rango $[0, 1]$ para mejorar la convergencia del optimizador.

```

1 X_train_flat = X_train_flat.astype(np.float32) / 255.0
2 X_test_flat = X_test_flat.astype(np.float32) / 255.0
3
4 # Alternativa: escalar a [-1, 1]
5 # X_train_flat = (X_train_flat.astype(np.float32) / 255.0) * 2 - 1

```

Listing 3: Normalización al rango $[0, 1]$

Paso 4: Codificación One-Hot de las etiquetas

Si se utilizan ciertas funciones de pérdida, las etiquetas deben codificarse en formato *one-hot* (por ejemplo, $5 \rightarrow [0, 0, 0, 0, 0, 1, 0, 0, 0, 0]$), para otras, las etiquetas enteras (0–9) pueden usarse directamente.

```

1 def one_hot_encode(labels, num_classes=10):
2     """Convierte etiquetas 0-9 en vectores one-hot de longitud 10."""
3     return np.eye(num_classes)[labels]
4
5 y_train_onehot = one_hot_encode(y_train)
6 y_test_onehot = one_hot_encode(y_test)
7
8 print(f"y_train_onehot: {y_train_onehot.shape}") # (60000, 10)
9 print(f"y_test_onehot: {y_test_onehot.shape}") # (10000, 10)

```

Listing 4: Codificación One-Hot de etiquetas

Paso 5: Visualizar imágenes

Una vez cargadas las imágenes como arrays NumPy, pueden graficarse directamente con matplotlib sin necesidad de convertirlas a ningún otro formato.

```

1 import matplotlib.pyplot as plt
2
3 # Visualizar una imagen individual
4 plt.imshow(X_train[0], cmap='gray')
5 plt.title(f"Etiqueta: {y_train[0]}")
6 plt.axis('off')
7 plt.show()
8
9 # Visualizar una grilla de 5x5 imagenes aleatorias
10 import numpy as np
11 indices = np.random.choice(len(X_train), 25, replace=False)
12
13 fig, axes = plt.subplots(5, 5, figsize=(8, 8))
14 for ax, idx in zip(axes.flat, indices):

```

```

15 ax.imshow(X_train[idx], cmap='gray')
16 ax.set_title(y_train[idx], fontsize=10)
17 ax.axis('off')
18 plt.suptitle("Muestra aleatoria de imagenes MNIST", y=1.02)
19 plt.tight_layout()
20 plt.show()

```

Listing 5: Visualización de imágenes MNIST

Nótese que `X_train` tiene forma $(60000, 28, 28)$: cada imagen es una matriz de 28×28 valores que `imshow` puede graficar directamente. El parámetro `cmap='gray'` asegura la escala de grises. **No es necesario convertir los archivos IDX a PNG ni a ningún otro formato de imagen.**

Consignas

1. **Carga y verificación del dataset.** Cargar las imágenes y etiquetas utilizando los pasos descritos en la guía anterior (o equivalente). Verificar que las dimensiones y los tipos de datos sean correctos antes de continuar.
2. **Análisis Exploratorio de Datos (EDA).**
 - a) Visualizar una grilla de imágenes aleatorias junto con sus etiquetas.
 - b) Analizar la distribución de clases (frecuencia de cada dígito 0–9) y determinar si el dataset está balanceado.
 - c) Visualizar el histograma de intensidad de los píxeles antes y después de la normalización.
3. **Preprocesamiento.** Aplicar los pasos de la guía: aplanamiento de imágenes ($28 \times 28 \rightarrow 784$), normalización de píxeles al rango $[0, 1]$ y codificación de etiquetas. Justificar la estrategia elegida para las etiquetas (*one-hot* vs. enteros).
4. **Diseño e implementación de la red neuronal.** Diseñar un MLP para clasificación multiclase (10 clases). Definir y justificar:
 - a) Número de capas ocultas, neuronas por capa y funciones de activación, incluyendo la capa de salida. Argumentar.
 - b) Función de costo apropiada para clasificación multiclase.
 - c) Optimizador y tasa de aprendizaje.
 - d) Estrategia de regularización para reducir el *overfitting*.
5. **Entrenamiento y curvas de aprendizaje.** Graficar las curvas de pérdida y de *accuracy* en entrenamiento y validación a lo largo de las épocas. Identificar el punto de *overfitting* si existiera.
6. **Evaluación del modelo.** Reportar *accuracy* global y por clase sobre el conjunto de test. Discutir si la métrica de *accuracy* es adecuada para este problema.
7. **Matriz de Confusión y análisis de errores.** Visualizar la Matriz de Confusión. Identificar los pares de dígitos que generan mayor confusión y explicar morfológi-

camente por qué esas confusiones tienen sentido. Seleccionar y mostrar al menos 5 imágenes clasificadas incorrectamente con alta confianza.

8. **Análisis y conclusiones.** Reflexionar sobre las limitaciones de los MLP frente a datos de imagen y cómo esas limitaciones podrían resolverse con otras arquitecturas.

3. Problema 3: Regresión

Objetivo

Construir un modelo de red neuronal que prediga el valor numérico de la rentabilidad (*Profit*) de una transacción comercial a partir de las características del pedido.

Dataset

- **Tipo de problema:** Regresión
- **Tamaño:** 9 994 transacciones $\times$ 21 características

Variable objetivo

Característica	Tipo	Descripción
Profit	Numérico (Float)	Beneficio neto de la venta. Puede ser negativo (pérdida) o positivo (ganancia).

Variables predictoras y accesorias

Característica	Tipo	Descripción
Identificadores de transacción (no predictivos)		
Row ID	Entero	Identificador secuencial.
Order ID	Categórico	ID único de pedido.
Customer ID	Categórico	ID de cliente. Alta cardinalidad; evaluar si incluir.
Variables temporales		
Order Date	Fecha (Cadena)	Fecha del pedido. Extraer año, mes, día y trimestre.
Ship Date	Fecha (Cadena)	Fecha de envío. Útil para calcular la demora.
Variables de envío y logística		
Ship Mode	Categórico	Método de envío.
Demografía del cliente		
Customer Name	Categórico	Nombre del cliente.
Segment	Categórico	Segmento de mercado.
Variables geográficas		
Country	Categórico	Constante en este dataset; poco informativo.
Region	Categórico	Región geográfica.
State	Categórico	Cardinalidad media.
City	Categórico	Alta cardinalidad. Evaluar tratamiento.

Característica	Tipo	Descripción
Postal Code	Entero	Alta cardinalidad. Evaluar si eliminar.
Variables de producto		
Category	Catégorico	Categoría principal del producto.
Sub-Category	Catégorico	Subcategoría del producto.
Product ID	Catégorico	ID de producto. Evaluar codificación.
Product Name	Catégorico	Redundante con categoría y subcategoría. Normalmente eliminar .
Detalles de la transacción (predictores principales)		
Sales	Numérico (Float)	Ingresos totales de la venta.
Quantity	Entero	Número de artículos pedidos.
Discount	Numérico (Float)	Tasa de descuento aplicada. Predictor crítico.

Consignas

1. Análisis Exploratorio de Datos (EDA).

- Visualizar la distribución de la variable objetivo `Profit`, incluyendo valores negativos y la presencia de *outliers*.
- Analizar la relación entre las características predictivas y la objetivo mediante gráficos de dispersión y matrices de correlación.
- Identificar variables con alta cardinalidad y determinar una estrategia para su tratamiento.

2. Preprocesamiento.

- Eliminar las columnas no predictivas.
- Extraer características útiles de las variables de fecha.
- Codificar las variables categóricas. Justificar la técnica elegida según la cardinalidad de cada variable.
- Escalar las variables numéricas **después** del split *train/test*. Argumentar qué escalador es más adecuado.

3. Diseño e implementación de la red neuronal. Diseñar un MLP para regresión. Definir y justificar:

- Función de activación en la capa de salida.
- Función de costo. Comparar MSE, MAE y Huber Loss en el contexto de la distribución de `Profit`.
- Número de capas ocultas, neuronas y funciones de activación.
- Optimizador, tasa de aprendizaje y estrategia de regularización.

4. Entrenamiento y curvas de aprendizaje. Graficar las curvas de pérdida de entrenamiento y validación. Identificar *overfitting* o *underfitting* e implementar una corrección.

5. **Evaluación del modelo.** Reportar las métricas sobre el conjunto de test e interpretarlas en términos del negocio (¿qué significa ese error en dólares?).
6. **Análisis de residuos.** Graficar el error de predicción ($y_{\text{real}} - y_{\text{pred}}$) en función de y_{real} . Determinar si el error es constante o si el modelo falla en pedidos de gran magnitud. Seleccionar y analizar los 5 casos con mayor error absoluto.
7. **Análisis y conclusiones.** Discutir si el costo computacional de un MLP se justifica frente a modelos alternativos para datos tabulares. Reflexionar sobre las limitaciones del modelo y propuestas de mejora.

4. Problema 4: Clasificación de Razas de Perros con CNN desde cero

Dataset Drive: [70 Dog Breeds Image Dataset - Drive](#)

Dataset Kaggle: [70 Dog Breeds Image Dataset - Kaggle](#)

Objetivo

Construir, entrenar y optimizar una red neuronal convolucional **desde cero** (sin *transfer learning*) para clasificar imágenes de razas de perros. El objetivo es experimentar sistemáticamente con distintas decisiones de diseño arquitectónico y de entrenamiento para maximizar el rendimiento del modelo.

Restricción: Queda **prohibido** el uso de modelos preentrenados o transfer learning en este ejercicio. Todos los pesos deben inicializarse aleatoriamente y entrenarse sobre el dataset.

Reducción del problema (recomendado)

Dado que el entrenamiento puede ser costoso en cómputo, se recomienda trabajar con un **subconjunto de 5 a 10 razas** elegidas por el grupo. Esto permite iterar más rápido sobre los experimentos. En el informe deben justificar la selección (pueden elegir razas visualmente similares para mayor dificultad, o variadas para mayor facilidad).

Checkpoints y reanudación

Se recomienda guardar el estado del modelo periódicamente durante el entrenamiento para poder retomarlo en caso de interrupción:

```

1 # Guardar checkpoint
2 torch.save({
3     'epoch': epoch,
4     'model_state_dict': model.state_dict(),
5     'optimizer_state_dict': optimizer.state_dict(),
6     'loss': loss,
7 }, 'checkpoint_epoch_{}.pth'.format(epoch))
8
9 # Retomar desde checkpoint
10 checkpoint = torch.load('checkpoint_epoch_N.pth')
11 model.load_state_dict(checkpoint['model_state_dict'])

```



Figura 1: Mosaico de imágenes de ejemplo del dataset *70 Dog Breeds Image Dataset*.

```
12 optimizer.load_state_dict(checkpoint['optimizer_state_dict'])  
13 start_epoch = checkpoint['epoch'] + 1
```

Listing 6: Guardado y reanudación desde checkpoint

Preparación de datos

- Aplicar **resize** a una resolución fija (ej. 128×128 o 224×224).
- **Normalizar** los píxeles (media y desviación estándar del conjunto de entrenamiento, o los valores estándar de ImageNet).
- Implementar un pipeline de **Data Augmentation** sobre el conjunto de entrenamiento (no sobre validación ni test).

Experimentos a realizar

Deberán documentar al menos **4 experimentos distintos**, organizados en una tabla comparativa. A continuación se sugiere una progresión lógica:

Experimento	Qué modificar
E1 — Línea base	CNN simple: 3 bloques Conv-BN-ReLU-MaxPool + cabeza clasificadora densa.
E2 — Arquitectura más profunda	Aumentar profundidad. Explorar el efecto de agregar más capas convolucionales y variar el tamaño de los filtros (3×3 vs 5×5) y los strides.
E3 — Bloques avanzados	Incorporar al menos uno de los siguientes: bloques residuales (skip connections estilo ResNet), módulos Inception (ramas paralelas con filtros de distinto tamaño y convoluciones 1×1 como cuello de botella).
E4 — Regularización y Data Augmentation	Agregar Dropout, BatchNormalization si no estaba, y un pipeline de Data Augmentation (flips horizontales, rotaciones, variaciones de brillo/contraste, recortes aleatorios, etc.).

Dentro de cada experimento también pueden variar hiperparámetros como learning rate, batch size, número de épocas y optimizador.

Qué reportar

Para cada experimento, completar la siguiente tabla:

Experimento	Acc. Train	Acc. Val	Acc. Test	# Parámetros	Épocas	Observaciones
E1						
E2						
E3						
E4						

Además, para el **mejor modelo** obtenido:

- Curvas de loss y accuracy (train vs. validación) por época.
- **Matriz de confusión** en el conjunto de test.
- Al menos 5 ejemplos de predicciones correctas e incorrectas con la imagen, la clase real y la clase predicha.
- Análisis breve: ¿qué razas se confunden más entre sí? ¿Por qué?

5. Modalidad de Entrega y Evaluación

El trabajo debe realizarse de manera grupal con exactamente **3 integrantes** (o 4 en caso de que la cantidad de estudiantes del curso no sea múltiplo de 3). Todo el material deberá almacenarse en un repositorio **GitHub privado**, al cual deberán agregar a los docentes como colaboradores.

Instancia	Fecha
Presentación del enunciado	martes 15 de septiembre
Entrega	viernes 9 de octubre, hasta las 23:59 hs
Defensas	lunes 19 de octubre y martes 20 de octubre

Fecha límite de entrega: viernes 9 de octubre, hasta las 23:59 hs. No se aceptará ningún trabajo entregado con posterioridad a dicha fecha y hora.

La entrega se realizará mediante un **correo electrónico con copia a ambos docentes**, con el siguiente formato:

- **Asunto:** AAIL - Entrega TP1 - Grupo X (reemplazar X por el número del grupo).
- **Cuerpo:** debe incluir:
 1. Número de grupo.
 2. Nombre completo de cada integrante.
 3. Link al repositorio de GitHub.

5.1. Notebooks

Cada problema debe resolverse en un **Jupyter Notebook separado**. El notebook debe cumplir los siguientes requisitos:

- El código debe organizarse en celdas según su propósito, con salidas claras (prints y visualizaciones) que documenten el resultado de cada proceso.
- Cada celda de código debe ir precedida de una celda Markdown con: **título** (usando jerarquía #, ##, ###), una breve descripción de lo que se realizará y las decisiones tomadas con su justificación.
- Cada celda de código debe ir seguida de una celda Markdown que explique el resultado obtenido y las conclusiones derivadas.
- El código debe incluir comentarios explicativos sobre conjuntos de líneas con un mismo propósito. Se deben usar nombres de variables descriptivos en notación `snake_case`.

5.2. Informe

El repositorio deberá contener un **informe en formato PDF** que incluya:

- Descripción del procedimiento seguido para resolver cada problema.
- Decisiones de diseño tomadas y sus justificaciones.
- Desafíos encontrados durante el desarrollo.
- Conclusiones sobre los aprendizajes obtenidos al diseñar y entrenar las redes neuronales para los distintos tipos de problema.

5.3. Defensa

La defensa se realizará de manera **virtual** en dos fechas consecutivas: **lunes 19 de octubre** y **martes 20 de octubre**. La fecha y hora de cada grupo será sorteada y comunicada con anticipación. El grupo deberá conectarse al enlace de videoconferencia en el horario asignado. **El integrante que no se presente a la defensa desaprobará el trabajo completo.** De no poder asistir en la fecha u horario sorteado, se puede negociar con otro grupo para intercambiar lugares.

La defensa consta de tres fases:

1. **Exposición grupal** (máximo 15 minutos): el grupo deberá presentar el desarrollo de las resoluciones a los problemas, las decisiones tomadas en cada problema y sus resultados. Se evaluará la capacidad de seleccionar los aspectos más relevantes y la gestión del tiempo. Se recomienda distribuir la exposición equitativamente entre los integrantes.
2. **Preguntas conceptuales individuales**: cada integrante responderá preguntas de carácter conceptual para terminar de definir la nota.
3. **Devolución general** por parte de los docentes.

Preferencia de día de defensa

Importante: quien tenga una preferencia por un día de defensa en particular (lunes 19 de octubre o martes 20 de octubre) deberá comunicarlo por **correo electrónico a ambos docentes antes de la fecha de entrega**. Los pedidos recibidos con posterioridad no serán tenidos en cuenta en el sorteo.

El correo deberá indicar obligatoriamente:

1. **Motivo** de la preferencia.
2. **Día preferido** para rendir la defensa.
3. **Carácter de la restricción:** si es **recurrente** o **eventual**. Es decir, si el impedimento se repite todas las semanas ese día (por ejemplo: “no puedo ningún lunes”) o si afecta únicamente a esa fecha puntual (por ejemplo: “no puedo ese lunes en particular”).

Las preferencias se considerarán en la medida de lo posible al armar el cronograma, priorizando las restricciones recurrentes y debidamente justificadas. Se buscará además mantener equilibrada la cantidad de grupos asignados a cada día.